

Implementando as redes neurais

Olá!

Nos vídeos anteriores desta aula aprendemos, basicamente, o funcionamento de uma rede neural o modo como ela computa uma informação.

Neste vídeo, iremos aprender a utilizá-lo a partir da biblioteca Scikit-learn e alguns dos cuidados especiais que devemos ter na sua implementação.

Então, iremos iniciar este vídeo importando a rede neural e algumas bibliotecas que utilizaremos no decorrer dele.

Primeiramente, iremos importar a própria rede neural: `from sklearn.neural_network import` e temos aqui alguns tipos de redes neurais diferentes, mas nós iremos utilizar aqui o "MLPRegressor", mas também poderíamos utilizar o "MLPClassifier" para uma classificação.

As duas funcionam de maneira bem similar alterando, apenas, o conjunto de dados. Iremos utilizar, também, o: `from sklearn.model_selection import train_test_split`.

"train_test_split" é a divisão de "train" e "test".

Na rede neural da linha anterior, o "import" é: "MLPRegressor".

Então, próxima linha: `from sklearn.preprocessing import MinMaxScaler`.

Iremos utilizar o "MinMaxScaler" para traduzir as nossas variáveis de entrada, as nossas features em valores entre 0 e 1.

E iremos importar, também, o nosso conjunto de "boston" de precificação das casas de Boston, então: `from sklearn.datasets import load_boston`.

Pronto.

Dessa maneira, nós temos os "import" para utilizar a rede neural.

Então, nesse caso, nós iremos ter uma utilização bem passo-a-passo de cada uma das nossas partes do que será executado para termos uma boa rede neural.

Então, vamos lá!

O primeiro passo que iremos fazer é carregar o nosso conjunto de dados.

Então, utilizaremos: `X,y = load_boston(return_X_y=True)`.

Dessa maneira, carregamos o nosso conjunto de dados.

Agora, iremos utilizar o nosso "train_test_split" iremos separar o conjunto de dados em "train" e "test".

Então, iremos utilizar a seguinte linha: `X_train, X_test, y_train, y_test = train_test_split(X,y)`.

Dessa maneira, teremos a divisão dos nossos dados em "train" e "test".

Então, até agora, nós carregamos o nosso conjunto de dados já separando os rótulos, a variável "y" e os atributos, a variável "X" e tivemos a divisão desses dados em "train" e "test".

A partir disso, nós precisamos, também tomar o cuidado de utilizar o "MinMaxScaler" porque a rede neural, pela maneira como ela é treinada a técnica de treinamento dela, que é o Backpropagation a técnica de retropropagação ela vai se beneficiar de variáveis em uma escala pequena e escalas parecidas.

Então, podemos utilizar o "MinMaxSplit" para transformar as nossas variáveis em uma escala menor.

Iremos utilizá-lo da seguinte maneira: `mm = MinMaxScaler()`.

A gente quebra a linha e utiliza tudo na mesma célula. Podemos utilizar: `X_train = mm.fit_transform(X_train)`. Assim, estamos transformando nossos dados com base nos atributos de "train".

Em seguida, quebramos a linha de novo e iremos utilizar, sem o fit: `X_test = mm.transform(X_test)`.

Dessa forma, estamos transformando os dados de "test" de acordo com os limites de "train".

Então, os nossos conjuntos de dados vão ter essa transformação do "test", de acordo com o "train".

Esta é uma técnica que utilizamos, de separar essa divisão para evitar o que chamamos de data leak que as informações de "train" sejam vazadas para o "test".

Executando esta linha, nós teremos que nossos dados foram transformados para uma escala menor.

Então, finalmente, podemos utilizar a rede neural.

Iremos utilizar aqui uma variável chamada "mlp" para guardar a rede: `mlp = MLPRegressor()` e, aqui, podemos ter alguns valores por exemplo, o regressor da rede neural nós podemos passar o número de neurônios e a quantidade de camadas, ou seja, o número de camadas e a quantidade de neurônios nelas.

Podemos passar, também, a função de ativação e alguns outros atributos, mas iremos nos concentrar principalmente nesses dois.

Então, como é ativação, por padrão, já está sendo utilizada a "ReLU".

A "ReLU" é interessante ao nosso caso porque, como comentei um pouco antes em um vídeo anterior ela vai pegar os valores negativos da saída do neurônio e irá transformá-los em 0, e o restante ela mantém como estava.

Isso é interessante porque o conjunto de dados "boston" se refere a valores de casas então, a "ReLU" é interessante porque ela não permite valores negativos.

Então, ela vai um bom resultado de toda maneira.

Com relação ao tamanho dos neurônios isso vai variar de problema para problema.

Como padrão, ele vai ter apenas uma camada intermediária como chamamos, uma camada curta de 100 neurônios.

O que seria uma camada curta?

Nós temos a camada de entrada, os valores de entrada do problema temos a primeira camada de neurônios, que é a primeira camada curta e a camada de saída, ou seja, o valor de saída é só um neurônio e ele funciona para nossa classificação e regressão.

Então, o que vamos definir é o número de camadas ocultas.

A saída e a entrada são definidas automaticamente não é necessário mexermos nelas.

Utilizando o SK-learn, iremos utilizar: `mlp = MLPRegressor(hidden_layer_sizes=(10,10,10), "hidden", de escondidos "_layer", de camada "_sizes", de tamanhos, tamanhos das camadas intermediárias. Depois, passamos a quantidade de camadas, separadas por vírgulas e o número de cada uma dessas camadas.`

Dessa maneira, temos três camadas, cada uma com 10 neurônios e podemos ir reajustando à medida que obtemos os resultados.

Lembrando: muitas camadas e neurônios podem gerar um modelo muito complexo e podemos ter um "overfitting" a partir disso.

Não queremos nem muitas camadas e neurônios nem poucas camadas e neurônios. Isso vai ser executado de acordo com a nossa necessidade.

Então, vamos até diminuir a quantidade de neurônios e utilizar: (10,5), ou seja, 10 neurônios na primeira camada oculta, e cinco neurônios na segunda camada oculta. Ficando assim: `mlp = MLPRegressor(hidden_layer_sizes=(10,5))`.

A partir disso, temos implementada a rede neural.

Agora, é treiná-la e obter o resultado.

Para treiná-la, utilizaremos, de maneira comum, o "fit": `mlp.fit(X_train,y_train)`.

Ao executar esta função temos que a rede neural foi treinada.

Aqui, na minha execução, apareceu um Warning, um aviso dizendo que a rede neural ainda poderia ser mais treinada ela ainda poderia convergir melhor com o resultado.

Mas vamos manter assim e, em outras execuções poderemos encontrar os valores, aumentando a quantidade de neurônios ou aumentando a quantidade máxima de interações que é um dos outros parâmetros do nosso regressor, da nossa "MLP".

Podemos utilizar, na próxima linha: `mlp.score(X_test,y_test)`.

E ao executar, temos um valor bem ruim que foi de "-1.0506...", ou seja, um valor bem ruim para os nossos dados.

Vamos tentar aumentar a quantidade de neurônios.

Aumentando a primeira camada para "100", e a segunda para "50" iremos verificar o que irá acontecer.

Ao executarmos a nossa rede neural aumentando as nossas quantidades de neurônios para "100" e "50" já tivemos uma melhora no resultado para 69% de acerto.

Vamos aumentar para duas camadas uma de "100", e outra de "100".

Ao executarmos, já subiu para 73%.

Vamos colocar uma terceira camada, com 50 neurônios já subiu para 79%.

Ou seja, neste caso aumentar a quantidade de neurônios está aumentando o nosso resultado até o ponto em que eu aumentei a última camada, de "50" para "100", ou seja, três camadas de 100 neurônios e o "score" não melhorou.

Podemos voltar a terceira camada para "50" aplicar uma nova camada de "50", e ver o que acontece.

Agora subiu para 80%.

Então, nós podemos ir mexendo nessas camadas, obtendo resultados e tentando entender o que está acontecendo com a rede.

À medida que vamos aumentando os neurônios e a capacidade da rede nós vamos tendo mais resultados positivos.

Outra das opções que podemos passar na "MLPRegressor" poderia ser a nossa quantidade máxima de interações.

Então, vou adicionar um novo atributo no "MLPRegressor", do lado do tamanho da rede que seria: `max_iter=1000`).

Ficando assim: `mlp = MLPRegressor(hidden_layer_sizes=(100,100,50,50), max_iter=1000)`.

O padrão é "200", mas usarei "1000", para fazer um teste.

A nossa rede demora um pouco mais para ser treinada.

Ao executar o nosso "score", já temos 82%.

Ou seja, a quantidade de interações de treinamento da rede aumentou e o nosso "score" também.

E, assim, podemos ir mexendo nos nossos parâmetros até encontrar uma rede que dê o melhor resultado possível sem que tenhamos overfitting.

Dessa maneira, nós encerramos a parte de implementação de redes neurais como utilizá-las, usando o Scikit-learn e eu recomendo que vocês possam experimentar diferentes variações da nossa rede neural para tentar obter os mais diferentes resultados e entender como a rede funciona.

Ela é muito prática, então, quanto mais nós mexemos nela mais podemos entender melhor como ela está funcionando.

Então, estamos encerrando esta aula por aqui. Agradeço pelo companheirismo na aula.

Até a próxima!